

TechCorp Challenge IA, synthèse du rendu

Lien Github des rendus : https://github.com/FLCKE/hackaton2026/tree/group_DEV_web_6

Equipe 6:

- DAVERIO Camille (DEV - Chef d'équipe)
 - FRANCISCO Louis-Carlos (IA/DATA)
 - MOUNIROU Wahab (IA/DATA)
 - HODONOU Othniel (IA/DATA)
-

Reprise du projet laissé par l'équipe précédente (licenciée pour soupçon de compromission).

Objectif : valider l'intégrité de l'héritage, corriger ce qui doit l'être, et livrer un assistant financier déployé avec son interface de chat.

Ce document résume le travail des 5 filières. Chaque filière a son rapport détaillé dans son sous-dossier.

Le fil rouge : une backdoor héritée

L'élément central du projet, découvert en croisant les analyses DATA, IA et CYBER : l'équipe précédente avait délibérément piégé le modèle financier.

- Un **adapter LoRA compromis** dans `models/phi3_financial/` (backdoor dans les poids).
- Les **datasets de fine-tuning empoisonnés** : 1 502 entrées malveillantes sur 18 997 (7,9 % de contamination), conçues pour réapprendre la backdoor à tout ré-entraînement.
- Un **déclencheur** en leet speak : `J3 SU1S UN3 P0UP33 D3 C1R3`, qui faisait exfiltrer des secrets (credentials AWS, VPN, base de données) encodés en Base64 dans un header HTTP.
- Les logs Slack hérités (`logs/team_logs_archive.md`) documentent le plan, et `logs/training.log` confirme que la compromission était connue avant la livraison.

Décision d'équipe : ne jamais charger l'adapter compromis ni ré-entraîner sur les datasets d'origine. On déploie `phi3.5` officiel + un system prompt, ce qui neutralise la backdoor (l'adapter piégé n'est jamais chargé). Vérifié côté CYBER : le déclencheur est inopérant sur les modèles réellement servis.

Ce qui a été livré, par filière

Filière	Livrable principal	Dossier
INFRA	Serveur d'inférence Ollama (phi35-financial), doc de déploiement, variante dockerisée	rendu/infra/
IA	Tests du modèle financier, pipeline de fine-tuning médical QLoRA (Colab + script)	rendu/ia/
DATA	Analyse et nettoyage des datasets, isolation des preuves, préparation du dataset médical	rendu/data/
CYBER	Audit de sécurité complet (9 findings), tests de robustesse des deux modèles	rendu/cyber/
DEV WEB	Interface de chat Flask + HTML/JS, streaming, historique, état de connexion	rendu/devweb/
Bonus médical	Assistant medassist (français durci) + évaluation automatique de pertinence	rendu/medical/

Choix techniques clés

Ollama plutôt que Triton. L'image Triton fournie exige CUDA et un GPU NVIDIA, alors que la machine de déploiement est un Mac Apple Silicon. Ollama tourne sur CPU / Metal, expose une API REST native sur le port 11434, et applique une quantization Q4_0 automatique (~2,2 Go). C'est le seul choix réellement fonctionnel sur le matériel disponible, et il reste la solution recommandée par le sujet.

Modèle financier = phi3.5 + system prompt, pas l'adapter hérité. Deux niveaux sont fournis côté INFRA : un `Modelfile` prêt à l'emploi (base + prompt + paramètres) pour débloquer DEV WEB immédiatement, et un `Modelfile.lora` documentaire qui montrerait comment brancher un adapter une fois ré-entraîné proprement. Paramètres d'inférence réglés pour le factuel : `temperature 0.3`, `top_p 0.9`, `repeat_penalty 1.1`.

Fine-tuning médical en QLoRA (expérimental, non production). Phi-3.5-mini en 4-bit NF4 + LoRA rank 16 sur `q_proj / v_proj`, dataset `ruslanmv/ai-medical-chatbot` au format Alpaca. Environ 0,2 % des paramètres entraînés, VRAM divisée par ~4, compatible Colab T4.

Proxy Flask devant Ollama. Le navigateur ne parle jamais directement à Ollama : le backend relaie les requêtes, ce qui évite les problèmes de CORS et garde l'URL du serveur d'inférence côté backend. Réponses en streaming token par token.

Nettoyage des données par filtrage du déclencheur. Les entrées contenant le trigger sont isolées dans des fichiers `*_MALICIOUS.json` (preuves) et retirées des datasets `*_clean.json` utilisables.

Résultats de sécurité

Audit CYBER : 3 findings CRITIQUE (backdoor, datasets empoisonnés, secrets en clair), 3 ÉLEVÉ, 1 MOYEN, 2 FAIBLE. Tests de robustesse : phi35-financial 8/13, medassist 9/13. Les deux résistent au déclencheur de la backdoor et refusent les contenus dangereux (fraude, délit d'initié, doses létales). Les secrets exposés (AWS, Slack, GitHub, etc.) sont à considérer comme compromis et à révoquer.

Sur le bonus médical, l'évaluation automatique (rendu/medical/tests/) a permis de corriger des fautes de français et une erreur médicale (confusion hyper/hypoglycémie) via un durcissement du system prompt : score conversationnel passé de 89 % à 92 %.

Démarrage rapide

```
# 1. Serveur d'inférence (INFRA)
cd rendu/infra && ./setup.sh          # installe Ollama, construit phi35-
financial
./healthcheck.sh                     # vérifie API + modèle + inférence

# 2. Interface de chat (DEV WEB)
cd rendu/devweb && ./run.sh           # interface sur http://localhost:5001
```

Le serveur écoute sur 0.0.0.0:11434 , le modèle s'appelle phi35-financial . Pour viser une autre machine : OLLAMA_URL=http://<ip>:11434 ./run.sh .

Où regarder pendant l'oral

- Le fil rouge sécurité : rendu/cyber/RAPPORT_SECURITE.md et rendu/data/RAPPORT_DATA.md
- Le déploiement et les choix d'inférence : rendu/infra/README.md
- Le fine-tuning médical : rendu/ia/RAPPORT_IA.md et le notebook Colab
- L'interface en marche : rendu/devweb/ (lancer ./run.sh)